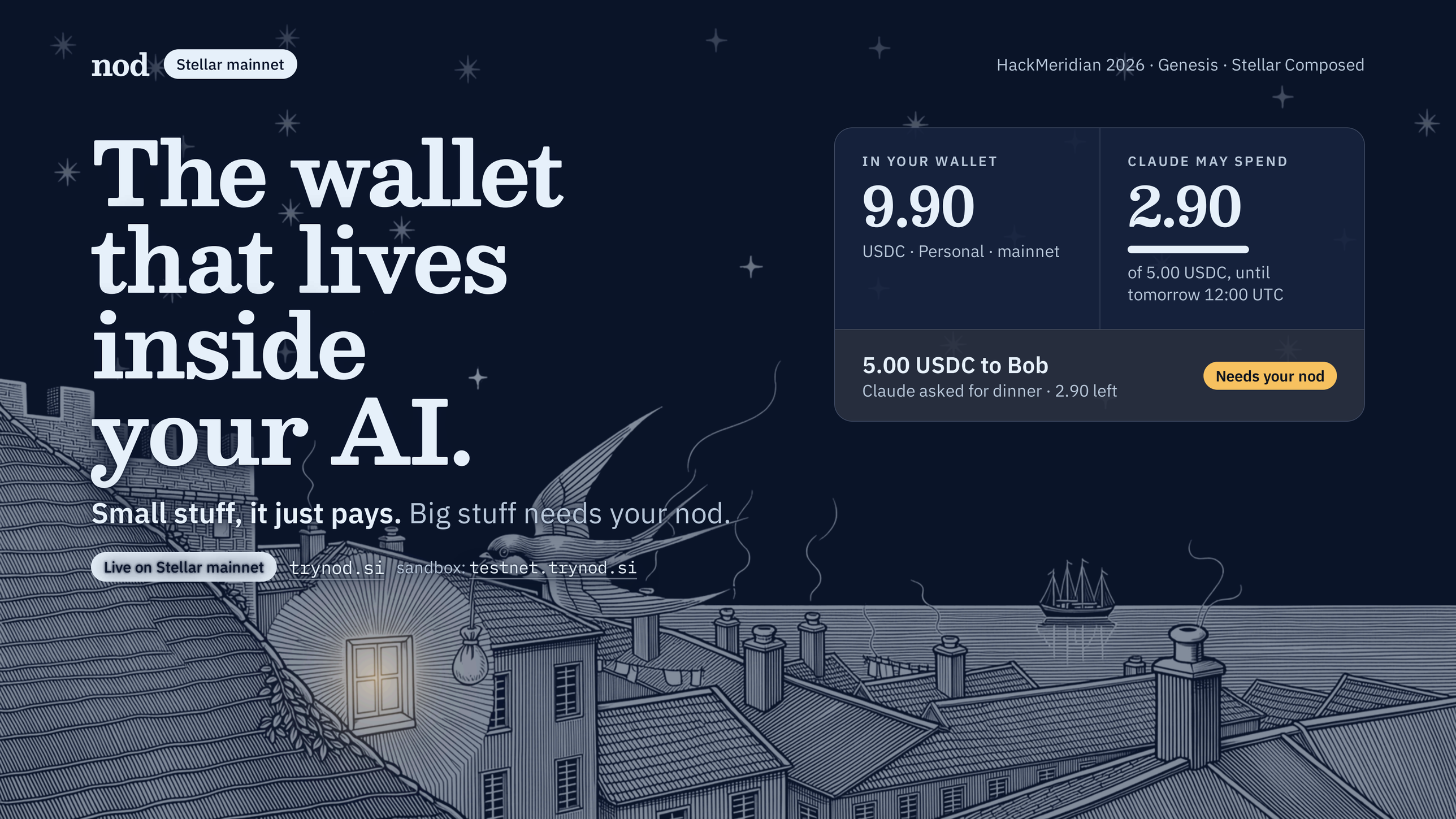
nod Stellar mainnet

HackMeridian 2026 · Genesis · Stellar Composed

The wallet that lives inside your AI.

Small stuff, it just pays. Big stuff needs your nod.

Live on Stellar mainnet trynod.si sandbox: testnet.trynod.si



WHY NOD

Your AI can do the work. Paying is still awkward.

CONSTANT INTERRUPTIONS

Every payment sends the task back to you.

A paid API result or a payment to a friend means leaving the conversation to finish the job yourself.

TOO MUCH ACCESS

A private key gives the AI access to your money.

You need a way to let it spend without trusting it with your whole balance.

> **“Set what your AI can spend. It asks for your passkey when it needs more.”**
>
> Nod puts the wallet in your chat. Stellar enforces the spending permission on your account.

---

nod Stellar mainnet USDC payments through the AI you already use

THE PRODUCT

Ask Claude to make a payment.

CONNECT YOUR WALLET

Add Nod to Claude with its connector URL. Choose what it can spend and when that permission expires. For this example: 5 USDC until tomorrow.

Small payments go through.

Claude pays for an API result or sends USDC to Alice. Its key signs the payment, and your account checks the allowance.

```
0.10 · market brief
2.00 · Alice → 2.90 left
```

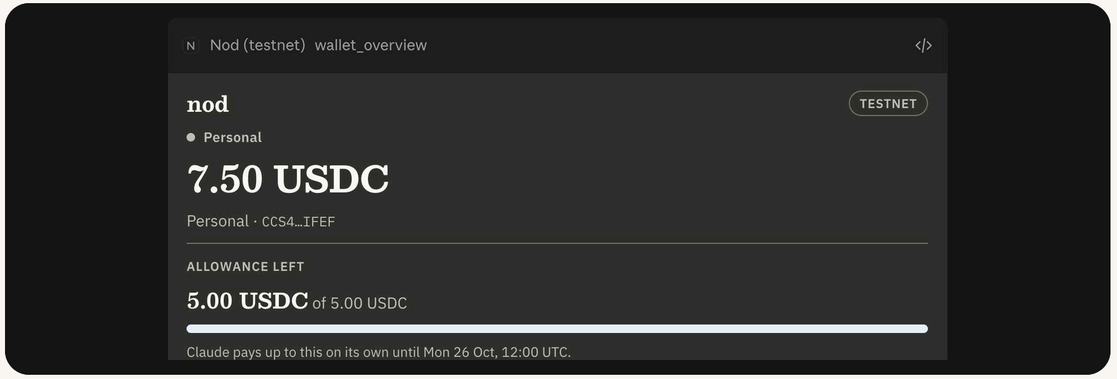
This payment needs you.

Claude opens an approval link with the recipient and amount. Your passkey signs that payment on Nod. It does not use the remaining allowance.

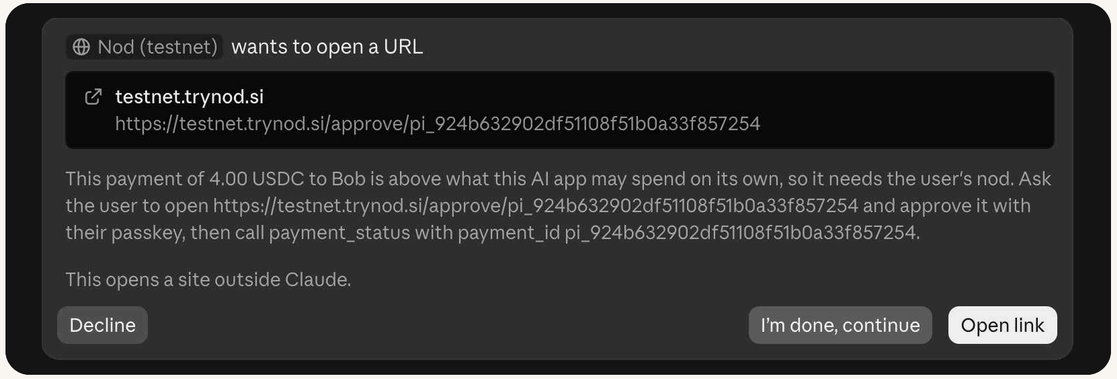
```
5.00 · Bob, dinner
passkey → paid
```

IN THE CONVERSATION

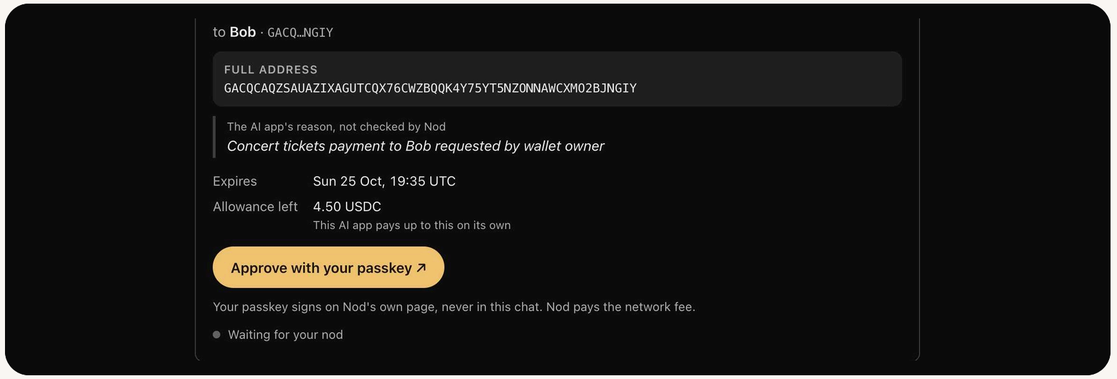
Your wallet is right there in the chat.



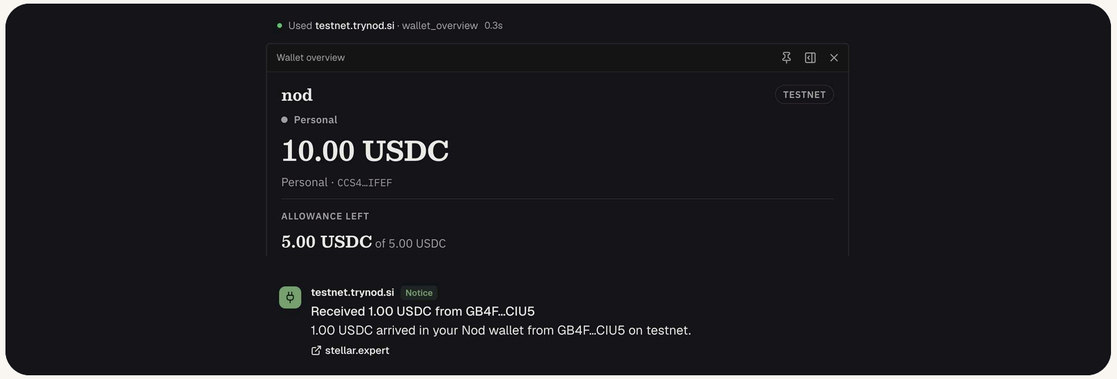
Claude shows your wallet and what it can spend



Claude opens Nod when a payment needs approval



ChatGPT developer-mode connector: waiting for passkey approval



Goblin receives a notification when money arrives (MCP Events)

HOW IT WORKS

The limit lives on your Stellar account.

Claude, ChatGPT, Goblin, any MCP app

OAuth 2.1 with PKCE; tools, cards (MCP Apps) and events. A connection alone spends nothing.

Nod wallet service

Cloudflare Worker. Holds each app's key, reserves before signing, re-simulates in enforcing mode, pays every fee.

Your OpenZeppelin smart account

```
rule 0 · your passkey · admin and approvals
rule n · Claude's key · ≤ 5.00 USDC until T
```

Circle USDC

Stellar Asset Contract, on mainnet and testnet

An allowance of 5 USDC permits exactly 5. One stroop more is rejected. — `c0f6f923`

testnet, spike S1

An over-limit transfer forced onchain is rejected by the account itself. — `3daf4b00`

testnet, error #3221, spending limit exceeded

A passkey approves a payment beyond the allowance without changing it. — `0339b8dd`

mainnet, 5 USDC to Bob with a WebAuthn signature

Revoke removes the rule from the account and sweeps the pocket back. — `e17e15d5`

mainnet; the sweep is ab6d493d

Nod pays every network fee. The owner holds 0 XLM. — `0.001 to 0.05 XLM`

per payment on mainnet, relayer as source

BUILT ON STELLAR

Stellar handles the permissions and payments.

OpenZeppelin smart account Each wallet has its own contract account	**OpenZeppelin spending-limit policy** The amount an app can spend before its permission expires	**Passkeys** WebAuthn verifier on the secp256r1 host function
Circle USDC Through the Stellar Asset Contract	**x402 facilitators** Public settlement through x402.org and OpenZeppelin Channels; tested with Stellar's mainnet demo seller	**MCP Apps and Events** Cards in the chat, notices by webhook, OAuth connector
SAC trust (CAP-73) Trustline and payment in one transaction	**Deposit router** SAC approve: a Stellar deposit address that sweeps USDC into the wallet; live on mainnet	**Sponsored reserves** Pockets hold no XLM; relayers pay every fee

> **“Agents that spend: Give an agent a wallet … set its limits with OpenZeppelin's policy library, and let it pay for APIs in stablecoins … over x402 or MPP.”**

Stellar Composed: agents that spend

> **“No seed phrases, no gas, no jargon. Spending limits … live in the wallet.”**

Stellar Composed, wallets idea

WHAT WE HAD TO SOLVE

Getting from a chat request to a payment.

THE POCKET MODEL

Paying through existing x402 facilitators.

Hosted facilitators do not yet accept smart-account payers. Claude's key funds a small pocket under the account policy, in 1 USDC steps, then pays a compatible Stellar x402 seller.

`your account` → policy → `pocket` → stock x402 client → `facilitator`

PERMISSION TO SPEND

Your passkey sets the allowance.

You choose an amount and expiry when connecting the app. Your passkey installs its rule on the account, and you can revoke that app later.

APPROVAL

You can still approve a larger payment.

Nod gives Claude an approval link. You see the recipient and amount on Nod's page, where your passkey signs the transaction.

CLAIM LINK

The recipient can start with a link.

Carol has a Stellar account but no USDC trustline. She accepts the link, and the trustline and payment settle together.

BUILT AT HACKMERIDIAN

I tested the wallet with my own money.

Started with the difficult parts

Before building the app, I used testnet spikes to check passkeys, account policies and x402 settlement. The tests and transaction records are in the repository.

Ran the flow on mainnet

In claude.ai, I bought an API result, paid Alice, approved Bob's payment and sent Carol a claim link. Revoking the connection removed its rule and returned the pocket balance.

Checked it in the apps people use

The wallet cards and approval links ran in Claude, ChatGPT developer mode and Goblin. Goblin and ChatGPT also received payment notifications.

Kept settlement interoperable

Public facilitators settled the x402 payments. Claude also paid Stellar's own mainnet demo endpoint, a seller I do not operate.

MARKET AND ECOSYSTEM

Reach people through the AI they already use.

Connect from your AI app

Nod connects over MCP. We tested it in Claude and Goblin, and in ChatGPT through developer mode. It is not listed in the ChatGPT app store.

Buyers for Stellar's x402 sellers

Stellar builders have shipped paid APIs and paid MCP tools. They need buyers whose owners will let them spend.

A payment brings the recipient in

On testnet, a recipient can open a link and create a passkey wallet. For a payment beyond the allowance, you approve a rule for that amount, valid seven days. Its key lives only in the link.

Live on mainnet

The full flow ran in claude.ai with my own wallet. Anyone can sign up and deposit Stellar USDC. Network fees in the mainnet run were 0.001 to 0.05 XLM per payment, paid by Nod.

TRY IT

Try Nod in your own Claude.

THE PRODUCT, ON MAINNET

`trynod.si`

THE SANDBOX: 10 TESTNET USDC PER NEW WALLET

`testnet.trynod.si/mcp`

OR WATCH THE SAMPLE AGENT

`testnet.trynod.si/sample`

"Buy the market brief"
0.10 USDC, no prompt; 4.90 left

"Send Alice 2 USDC"
paid on its own; 2.90 left

"Pay Bob 5 USDC for dinner"
More than what is left, so it needs your nod

Full steps in evidence/JUDGE-TEST-PATH.md.

What's next

On mainnet: links that create wallets and AI-added contacts (live on testnet)

Recovery passkey and shared wallets

Cash-out through a Stellar anchor

Direct x402 payments once facilitators accept smart accounts

Follow up uncertain payments and clean up expired permissions

